4. Gyakorlat

Python kurzus

Függvények

1.feladat: Adott egy mátrix: mx = [[1,2,3], [4,5,6], [7,8,9]] Listagenerátorral írd ki a mátrix elemeit egymás után sorba (listaként).

2.feladat: Írd ki az (x,y) koordinátákat listagenerátorral, ahol az x a [0,1], és az y a [0,2] zárt intervallum egészei legyenek.

3.feladat: Adottak az alábbi fájlnevek egy listában: files = ["py.py", "py.py.txt", "hello.docx", "music.json", "names.txt", "doctor_x.xlsx", "abc.json"]

a) Készítsünk egy listát, amely csak a .py kiterjesztésű fájlokat tartalmazza.
b) Gyűjtsük össze milyen különböző kiterjesztései vannak a fájloknak.
c) Számoljuk meg, hogy az egyes kiterjesztésekből hány darab van a listában.
d) Csoportosítsuk, majd írassuk ki a fájlneveket kiterjesztések alapján.

Alternatív másolási módok

lista = [1, 2, 3]

Referencia hozzárendelés (alias létrehozása)

```
lista1 = lista
print(lista1 is lista)          # True
print(id(lista1)==id(lista))   #True
```

Shallow copy

1. .copy() metódus
```
másolat1 = lista.copy()
```
2. Szeletelés
```
másolat2 = lista[:]
```
3. list() konstruktor
```
másolat3 = list(lista)
```

Deep copy (beágyazott objektumokhoz)

```
import copy
mély_másolat = copy.deepcopy(lista)
```

Shallow copy probléma

Beágyazott objektumoknál (listák, szótárak), a referenciák másolódnak, nem maguk az objektumok:

```
lista = [[1, 2], [3, 4]]
másolat = lista.copy()
másolat[0].append(3)          # A beágyazott lista módosítása
print(lista)                  # [[1, 2, 3], [3, 4]]   # változott!
print(másolat)                # [[1, 2, 3], [3, 4]]
```

Függvények bevezetés, paraméterátadás

```
def hi():
    print("hi!")
hi()
```

```
def hi():
    print("hi")
hi(5)
```

```
def hi():
    return "Hi!"
print(hi())
```

```
def hi():
    return
    print("Hi!")
hi()
```

```
def hi():
    print("hi!")
    return "Hi!"
print(hi())
```

```
def szorzat(a,b,c):
    print(a * b * c)
szorzat(3,4,3)
```

```
def szorzat(a,b,c):
    print(a * b * c)
szorzat(a=2,c=3,b=4)
```

```
def szorzat(a=1,b=1,c=1):
    return a * b * c
print(szorzat())
```

```
def szorzat(a=1,b=1,c=1):
    return a * b * c
print(szorzat(c=3,b=4))
```

```
def szorzat(a=1,b=1,c=4):
    return a * b * c
print(szorzat(2,3))
```

```
def szorzat(a=1,b=1,c=1):
    return a * b * c
print(szorzat(1,3,4,5))
```

Pozicionális (*args) és kulcsszó-argumentumok (**kwargs)

- ***args – pozicionális argumentumok**
  - A * jelöli azt, hogy a függvény **tetszőleges számú** pozicionális argumentumot fogadhat el.
  - Az argumentumok **tuple**-ként kerülnek tárolásra a függvényen belül

- ****kwargs - kulcsszó-argumentumok (keyword arguments)**
  - **Tetszőleges számú** kulcsszó-argumentumot adhatunk át egy függvénynek.
  - A kulcsszó-argumentumok név=érték párokként kerülnek átadásra.
  - A **kwargs a kulcsszó-argumentumokat **szótárként (dict)** tárolja, ahol a kulcsok az argumentumok nevei, az értékek pedig az ezekhez tartozó értékek.

Példák

```
def összeg(*args):
    return sum(args)

print(összeg(1, 2, 3))
print(összeg(5, 10, 15, 20))
```

```
6
50
```

```
def profil(**kwargs):
    for key, val in kwargs.items():
        print(f"{key.capitalize()}: {val}")

profil(név="Anna", kor=20, város="Sopron")
```

```
Név: Anna
Kor: 30
Város: Sopron
```

```
def mindent_kezel(name, *args, **kwargs):
    print(f"Név: {name}")
    print("Pozicionális argumentumok:", args)
    print("Kulcsszó-argumentumok:", kwargs)
mindent_kezel("Anna", 10, 20, 30, város="Sopron", kor=20)
```

```
Név: Anna
Pozicionális argumentumok: (10, 20, 30)
Kulcsszó-argumentumok: {'város': 'Sopron', 'kor': 20}
```

yield utasítás – generátor függvény

- A normál függvényeknél a **return** egyszeri értéket ad vissza és megszakítja a függvény futását
- A **yield** visszaad egy értéket és nem fejezi be a függvény végrehajtását

Generátor objektum használata:

```
def függvény(n):
    for i in range(n):
        yield i

# 1. Lista konverzió:
gen = függvény(3)
print(list(gen))  # [0, 1, 2]
```

```
# 2. Iterálás:
gen = függvény(3)
for érték in gen:
    print(érték)  # 0, 1, 2

# 3. Next() használat:
gen = függvény(3)
print(next(gen))  # 0
print(next(gen))  # 1
print(next(gen))  # 2
# print(next(gen))  # StopIteration error
```

Példák:

```
def függvény(n):
    for i in range(n):
        return i
for v in range(5):
    print(függvény(v))
```

```
None
0
0
0
0
```

```
def függvény(n):
    for i in range(n):
        yield i
for v in függvény(5):
    print(v)
```

```
0
1
2
3
4
```

```
def négyzetszámok(n):
    for i in range(n):
        négyzet = (i+1)**2
        yield négyzet

for elem in négyzetszámok(10):
    print(elem)
```

```
1
4
9
16
25
36
49
64
81
100
```

Fibonacci sorozat:

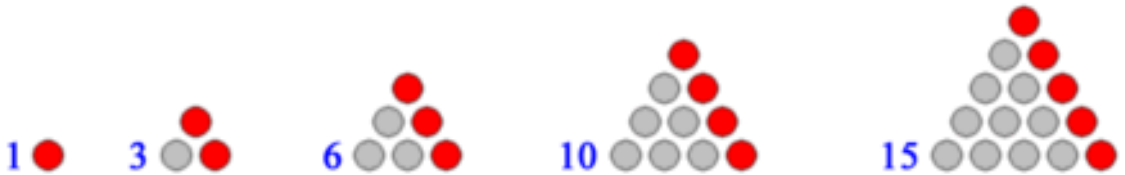
f(1) = f(2) = 1 és

f(n) = f(n-1) + f(n-2) ha n >2

```
def fibonacci(n):
    a = b = 1
    for i in range(n):
        if i == 0:
            yield 1
        else:
            a, b = b, a+b
            yield a
fibs= list(fibonacci(10))
print(fibs)
```

[1, 1, 2, 3, 5, 8, 13, 21, 34, 55]

Háromszögszámok:




```
def háromszögszámok(n):
    s = 0
    for i in range(1,n+1):
        s += i
        yield s
for elem in háromszögszámok(10):
    print(elem)
```

```
1
3
6
10
15
21
28
36
45
55
```

Lambda függvények:

lambda argument1, argument2, ... : kifejezés

```
print((lambda a, b: a + b)(2,5))      #összeadás        7
print((lambda a, b: a - b)(2,5))      #kivonás          -3
print((lambda a, b: a * b)(2,5))      #szorzás          10
print((lambda a, b: a ** b)(2,5))     #hatványozás      32
```

```
print((lambda a,b: (a**2 + b**2)**0.5)(3,4))   #Pitagorasz-tétel   5.0
```

```
import random
lista = [random.randint(1, 100) for i in range(10)]
print(lista)
print(list(filter(lambda x: x % 2 == 0, lista)))
```

```
[26, 99, 74, 53, 82, 53, 66, 93, 95, 79]
[26, 74, 82, 66]
```

Példák:

```
import random

def függvény(args, szabály):
    for x in args:
        print(szabály(x), end=' ')
    print()

lista = [random.randint(1, 100) for i in range(10)]
print(lista)

függvény(lista, lambda x: x % 3)
függvény(lista, lambda x: x % 5)
függvény(lista, lambda x: x + 1)
```

```
[5, 64, 70, 63, 99, 75, 80, 82, 80, 48]
2 1 1 0 0 0 2 1 2 0
0 4 0 3 4 0 0 2 0 3
6 65 71 64 100 76 81 83 81 49
```

```
számok = [1, 2, 3, 4, 5]
négyzetek = list(map(lambda x: x ** 2, számok))
print(négyzetek)
```

```
[1, 4, 9, 16, 25]
```

```
def fv_érték(args, polinom):
   for x in args:
      print('f(', x,')=', polinom(x), sep='')

fv_érték([x for x in range(-2,4)], lambda x: 2 * x**2 - 4 * x + 2)
```

```
f(-2)=18
f(-1)=8
f(0)=2
f(1)=0
f(2)=2
f(3)=8
```

```
data = [random.randint(-10,10) for x in range(10)]

filtered = list(filter(lambda x: x > 0 and x % 2 == 0, data))

print(data)
print(filtered)
```

```
[9, 10, -8, -2, -5, -2, 0, 1, 2, -7]
[10, 2]
```

Feladatok:

1. Írj egy függvényt, ami egy fokszámot radiánba vált át! Használd a math modult és a .radians() metódust.

2. A Jaccard-index egy olyan index, ami két halmaznak a hasonlóságát méri. Implementáld ezt a függvényt a képlet alapján.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

3. Implementáld a Quicksort-ot listagenerátorokkal, lambda függvényként egy sorban. Használj rekurziót.
Példa lista: [4214,214,215,1,2,51,51,56,1,5543,66,34,5234,52322]

```
# qs = lambda lst: [rekurzív_hívás] if [feltétel] else [alap_eset]    →    if lst else []
lst[0]      # Az első elem legyen a pivot
qs([x for x in lst[1:] if x <= lst[0]])     # Particionálás, bal oldal, rekurzió
[lst[0]]     # A pivot listaként
qs([x for x in lst if x > lst[0]])          # Particionálás, jobb oldal, rekurzió
# Összefűzés, bal_oldal + [pivot] + jobb_oldal
```

Gyakorlás, házi feladat: Feladatok_4_lambda_fv.py

```
# 1. Feladat: Lambda két szám szorzására
# Bemenet: 2, 3  # Kimenet: 6

# 2. Feladat: Lambda két szám maximumának megtalálására
# Bemenet: 2, 3  # Kimenet: 3

# 3. Feladat: Lambda egy szám páros voltának ellenőrzésére
# Bemenet: 4    # Kimenet: True

# 4. Feladat: Lambda egy string megfordítására
# Bemenet:  "hello"     # Kimenet: "olleh"

# 5. Feladat: Lambda egy szám négyzetének kiszámítására
# Bemenet: 4     # Kimenet: 16

# 6. Feladat: Lambda páros számok szűrésére egy listából
# Bemenet: [1, 2, 3, 4, 5, 6]))   # Kimenet: [2, 4, 6]

# 7. Feladat: Lambda stringek listájának nagybetűssé alakítására
# Bemenet: ["hello", "world"]))    # Kimenet: ["HELLO", "WORLD"]
```

8. Lambda tuple-ök listájának rendezésére a második elem alapján
Bemenet: [(1, 3), (2, 2), (3, 1)] # Kimenet: [(3, 1), (2, 2), (1, 3)]

9. Lambda stringek hosszának megtalálására egy listában
Bemenet: ["hello", "world"] # Kimenet: [5, 5]

10. Feladat: Lambda egy konstans hozzáadására minden elemhez egy listában
Bemenet: [1, 2, 3], 5 # Kimenet: [6, 7, 8]

11. Feladat: Lambda két lista metszetének megtalálására
Bemenet: [1, 2, 3], [2, 3, 4] # Kimenet: [2, 3]

12. Feladat: Lambda annak ellenőrzésére, hogy egy string csak számjegyeket tartalmaz-e
Bemenet: "123" # Kimenet: True

13. Feladat: Lambda duplikátumok eltávolítására egy listából
Bemenet: [1, 2, 2, 3, 4, 4, 5] # Kimenet: [1, 2, 3, 4, 5]

14. Feladat: Lambda egy szám faktoriálisának kiszámítására
Bemenet: 5 # Kimenet: 120

15. Feladat: Lambda annak ellenőrzésére, hogy egy string palindrom-e
Bemenet: "racecar" # Kimenet: True

Elemezd a kódot:

```
def pizza_készítés(méret, *feltét, **extra):

    print(f"Készítünk egy {méret} méretű pizzát a következő feltétekkel:")
    for elem in feltét:
        print(f"- {elem}")

    if extra.get("extra_sajt"):
        print("Extra sajtot teszünk rá!")

    if extra.get("glutén_mentes"):
        print("Gluténmentes tésztát használunk.")

pizza_készítés("nagy", "sonka", "gomba", extra_sajt=True, glutén_mentes=True)
```

Elemezd a kódot:

```
def rendelés_összegzés(**a):
    print("Rendelés részletei:")
    for kulcs, érték in a.items():
        print(f"{kulcs.capitalize()}: {érték}")

    if "fizetés" not in a:
        print("Figyelem! Nincs fizetési mód megadva.")
    if a.get("ingyen_szállítás", False):
        print("Az ingyenes szállítás elérhető.")
    else:
        print("Nincs ingyenes szállítás.")
rendelés_összegzés(termék="Laptop", ár=350000, darab=1, fizetés="bankkártya",
ingyen_szállítás=True)
```

Házi feladat: Pascal háromszög

Ha az adatszerkezet sorok listáinak a listája, akkor a háromszög minden új sora egy olyan új lista, amelyet 1-gyel kezdünk és fejezünk be, és az új sor belső elemeinek számolása az előző sor elemeiből történik:

uj_sor[i] = haromszog[sor - 1][i - 1] + haromszog[sor - 1][i]

```
Add meg, hány soros Pascal-háromszöget szeretnél generálni: 8
            1
           1  1
          1  2  1
         1  3  3  1
        1  4  6  4  1
      1  5  10  10  5  1
    1  6  15  20  15  6  1
  1  7  21  35  35  21  7  1
```

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00